ESCOLA SENAI “GASPAR RICARDO JÚNIOR”

RYAN AMORIM CARVALHO

RELATÓRIO TÉCNICO
REACT.JS — CARACTERÍSTICAS, VANTAGENS E APLICAÇÃO EM UM DASHBOARD DE CONTROLE DE ALMOXARIFADO

Relatório Técnico apresentado como requisito parcial de avaliação na disciplina de Frameworks Front-end no curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas.

Professor: Prof. Me. Deivison S. Takatu

Sorocaba — SP
10 de novembro de 2026

1. INTRODUÇÃO

O desenvolvimento de aplicações Web passou por diversas transformações ao longo dos últimos anos. Com o crescimento dos sistemas digitais, tornou-se necessário desenvolver interfaces cada vez mais dinâmicas, interativas, responsivas e de fácil manutenção. Nesse cenário, surgiram bibliotecas e frameworks que auxiliam os desenvolvedores na construção de aplicações modernas e altamente performáticas.

Entre as principais tecnologias utilizadas no desenvolvimento Front-end está o React.js, uma biblioteca JavaScript criada para facilitar a construção de interfaces de usuário baseadas no conceito de programação reativa. Seu principal diferencial está na utilização de componentes reutilizáveis, permitindo dividir uma aplicação complexa em partes menores, organizadas, testáveis e totalmente independentes.

O React foi originalmente concebido e desenvolvido pela equipe de engenharia do Facebook (atualmente Meta) para resolver problemas internos de sincronização de dados na interface da rede social, sendo disponibilizado publicamente como projeto de código aberto em 2013. Desde então, tornou-se a tecnologia dominante e amplamente adotada na indústria mundial de desenvolvimento de software Web, contando com uma comunidade extremamente ativa e um vasto ecossistema de ferramentas complementares.

O objetivo principal deste relatório técnico é apresentar em profundidade as características fundamentais do React.js, suas vantagens arquiteturais, limitações práticas e aplicações reais no mercado de tecnologia. Além da base teórica, o trabalho apresenta um exemplo prático de aplicação através da modelagem e construção de um Dashboard de Controle de Almoxarifado voltado à gestão de materiais de escritório.

O sistema proposto tem como finalidade demonstrar de forma clara como os conceitos de componentização, controle de estado local (Hooks), transmissão de dados (Props) e renderização condicional se articulam no desenvolvimento de uma interface administrativa robusta, capaz de gerenciar estoques, cadastrar novos materiais e monitorar movimentações em tempo real.

2. CONCEITO E PRINCIPAIS CARACTERÍSTICAS DO REACT

O React é classificado primariamente como uma biblioteca JavaScript de código aberto voltada para a construção de interfaces de usuário (UI). Diferentemente de frameworks opinativos completos (como Angular ou Ember) que determinam a estrutura exata do projeto e incluem ferramentas nativas para roteamento, formulários e requisições HTTP, o

React concentra-se estritamente na camada de visualização (View), oferecendo máxima flexibilidade técnica aos desenvolvedores.

A pilar central do React reside no modelo de programação declarativo combinado com a arquitetura orientada a componentes. Nesse modelo, o desenvolvedor descreve como a interface deve se parecer em determinado estado, e o React encarrega-se de atualizar de forma eficiente a árvore do DOM (Document Object Model) real quando ocorrem alterações nos dados da aplicação.

Em aplicações administrativas complexas, como um dashboard corporativo, a interface é decomposta em blocos granulares: cabeçalhos de navegação, painéis laterais (sidebars), cartões de indicadores (KPIs), tabelas dinâmicas de produtos e formulários de cadastro. Cada um desses blocos é isolado e encapsulado em seu próprio componente.

2.1 Componentização

A componentização promove uma organização de código significativamente superior, facilitando a manutenibilidade, a reutilização de elementos de UI e o trabalho colaborativo em equipes multidisciplinares.

No React moderno, um componente é tipicamente construído como uma função JavaScript pura que retorna uma estrutura de interface descrita em JSX. A abaixo ilustra a criação de um componente simples responsável por exibir o cartão de métrica de produtos cadastrados:

```
function CardEstoque() {
  return (
    <div className="card">
      <h3>Produtos cadastrados</h3>
      <p>120</p>
    </div>
  );
}

export default CardEstoque;
```

Conforme demonstrado no trecho de código anterior, o componente encapsula tanto a marcação quanto a estilização e a lógica local, podendo ser importado e instanciado quantas vezes forem necessárias dentro da aplicação.

2.2 JSX (JavaScript XML)

O JSX é uma extensão de sintaxe para JavaScript que permite escrever estruturas semelhantes ao HTML diretamente dentro do código do componente. Embora a sintaxe lembre o HTML tradicional, o JSX é transformado durante o processo de compilação (via Babel ou ESBuild) em chamadas de função estritas JavaScript, especificamente `React.createElement()`.

A utilização do JSX simplifica drasticamente a construção de interfaces visuais, pois permite interpolar expressões, variáveis e funções JavaScript diretamente na estrutura dos elementos HTML através do uso de chaves `{}`.

```
const produto = "Papel A4";

function Produto() {
  return <h2>{produto}</h2>;
}

export default Produto;
```

No exemplo acima, a constante `produto` é injetada de forma reativa na marcação visual. Essa integração fluida entre lógica de programação e estrutura visual elimina a necessidade de manipulação manual e imperativa do DOM via scripts externos.

2.3 Props e Reutilização de Componentes

As propriedades (Props, abreviação de *properties*) constituem o mecanismo primário de comunicação unidirecional no React. Elas permitem que dados e funções sejam passados de um componente pai para um componente filho, tornando os componentes altamente dinâmicos e genéricos.

Dessa forma, em vez de criar múltiplos componentes estáticos para diferentes indicadores no dashboard, cria-se um único componente genérico reutilizável que aceita parâmetros customizados via Props:

```
function Indicador({ titulo, valor }) {
  return (
    <div className="card">
      <h3>{titulo}</h3>
      <strong>{valor}</strong>
    </div>
  );
```

```
}

export default Indicador;
```

Ao instanciar o componente `<Indicador titulo="Saídas de Estoque" valor={45} />`, o React renderiza os valores passados de forma isolada, garantindo total reuso de código e mantendo a consistência visual em todo o sistema.

2.4 Hooks e Gerenciamento de Estado

Introduzidos na versão React 16.8, os Hooks revolucionaram o desenvolvimento Front-end ao permitir que componentes funcionais utilizem estado interno e métodos do ciclo de vida sem a necessidade de escrever classes complexas.

O Hook mais fundamental é o `useState`, que declara uma variável de estado mantida pelo React entre as renderizações da página e disponibiliza uma função modificadora encarregada de disparar a re-renderização automática do componente quando o valor é alterado.

```
import { useState } from "react";

function Contador() {
  const [quantidade, setQuantidade] = useState(0);

  return (
    <div>
      <p>Quantidade: {quantidade}</p>
      <button onClick={() => setQuantidade(quantidade + 1)}>
        Adicionar
      </button>
    </div>
  );
}

export default Contador;
```

Quando o usuário aciona o evento de clique no botão `Adicionar`, a função `setQuantidade` atualiza o estado interno e sinaliza ao motor do React que a interface precisa ser re-renderizada com o novo valor refletido na tela. Essa reatividade é vital para dashboards administrativos operando com dados dinâmicos.

3. VANTAGENS E LIMITAÇÕES DO REACT

Como qualquer decisão arquitetural no desenvolvimento de software, a adoção do React.js envolve a análise ponderada de seus pontos fortes e de suas restrições operacionais.

3.1 Principais Vantagens

• **Reutilização de Código:** A componentização permite criar bibliotecas internas de UI (Design Systems), reduzindo significativamente o tempo de desenvolvimento em projetos subsequentes.

• **Organização e Manutenibilidade:** A divisão em componentes menores isola falhas, facilita testes unitários e melhora exponencialmente a legibilidade do projeto.

• **Desempenho Otimizado:** O Virtual DOM minimiza alterações diretas na árvore do navegador, atualizando apenas os nós estritamente necessários para alta performance.

• **Ecossistema Maturo:** Amplo suporte a bibliotecas de roteamento (React Router), gerenciamento de estado global (Redux, Zustand) e estilização (Tailwind, Styled Components).

• **Suporte da Comunidade:** A enorme comunidade global assegura documentação atualizada, pacotes de terceiros consolidados e facilidade no recrutamento de profissionais qualificados.

3.2 Limitações e Desafios

• **Curva de Aprendizado Inicial:** Conceitos como fluxo de dados unidirecional, imutabilidade de estado, JSX e Hooks exigem mudança de paradigma em relação ao desenvolvimento Web tradicional.

• **Arquitetura Não Opinativa:** Por ser uma biblioteca focada apenas na View, o React exige que a equipe escolha e configure individualmente bibliotecas de roteamento, formulários e requisições HTTP.

• **Risco de Complexidade Excessiva:** Se não houver convenções rígidas de projeto, aplicações React podem sofrer com proliferação desordenada de componentes, *prop drilling* e renderizações desnecessárias.

4. APLICAÇÕES DO REACT NO MERCADO

O React consolidou-se como a biblioteca de Front-end mais adotada pelas maiores empresas de tecnologia do mundo. Sua capacidade de lidar com dados em tempo real e interfaces altamente interativas faz dele a escolha ideal para diversos domínios da indústria:

1. Dashboards e Painéis Administrativos: Sistemas corporativos com múltiplos módulos de relatórios, gráficos em tempo real e controle de acesso granular exigem a reatividade e a modulação oferecidas pelo React.

2. Plataformas de E-commerce: Plataformas como Amazon, Shopify e Shopee utilizam React para fornecer experiências de compra fluidas, com atualização instantânea de carrinhos e catálogos.

3. Redes Sociais e Streaming: Plataformas como Netflix, Twitter/X e Meta dependem de arquiteturas reativas para lidar com feeds dinâmicos, notificações instantâneas e execução de mídias.

4. Aplicações Financeiras e Fintechs: Bancos digitais e fintechs aproveitam o ecossistema React para criar painéis financeiros seguros, dinâmicos e com alta precisão de dados.

A relevância do React também é ratificada pelas pesquisas anuais da comunidade de desenvolvedores. No Stack Overflow Developer Survey 2025, o React manteve-se na liderança entre os frameworks e bibliotecas Web mais procurados e utilizados por profissionais no mundo todo.

5. EXEMPLO PRÁTICO: DASHBOARD DE CONTROLE DE ALMOXARIFADO

Para ilustrar concretamente a aplicação dos conceitos teóricos apresentados, propõe-se a implementação de um Dashboard de Controle de Almoxarifado voltado para a gestão de materiais de escritório em um ambiente corporativo ou educacional.

5.1 Objetivo e Escopo da Aplicação

O sistema visa substituir planilhas manuais e registros descentralizados por uma interface moderna, centralizada e responsiva. Através dela, o gestor de almoxarifado pode acompanhar os níveis de estoque em tempo real, cadastrar materiais e registrar movimentações de entrada e saída.

5.2 Funcionalidades Principais

• **Painel Geral de Indicadores (KPIs):** Exibição resumida do total de itens cadastrados, total de unidades em estoque, alertas de itens críticos e contagem de movimentações do dia.

• **Gestão e Cadastro de Produtos:** Tabela dinâmica listando os produtos com filtros por categoria, busca textual e indicação visual de status.

• **Registro de Entradas:** Mecanismo para registrar reposições de lote de materiais, incrementando automaticamente o saldo total do item.

• **Registro de Saídas:** Mecanismo para registrar retiradas por departamentos, com validação de saldo mínimo para evitar estoques negativos.

• **Alerta de Estoque Crítico:** Notificações visuais destacando materiais cuja quantidade atual esteja abaixo do limite mínimo cadastrado.

5.3 Produtos Utilizados como Exemplo

A aplicação demonstra a gestão com os seguintes itens típicos de escritório:

• Papel A4 75g (Caixa com 10 pacotes) — Categoria: Papelaria

• Caneta Esferográfica Azul (Caixa com 50 unid) — Categoria: Escrita

• Cartucho de Tinta Preto HP — Categoria: Suprimentos de Impressão

• Bloco de Notas Adesivas 76x76mm — Categoria: Papelaria

5.4 Estrutura de Arquivos do Projeto

A arquitetura de pastas do projeto segue as melhores práticas da comunidade React, garantindo separação clara de responsabilidades entre componentes de UI, páginas e utilitários de dados:

```
src/
├── components/
│   ├── Sidebar.jsx          # Navegação lateral
│   ├── Header.jsx           # Cabeçalho com perfil do usuário
│   ├── StatCard.jsx         # Cartões de métricas e KPIs
│   ├── ProductTable.jsx     # Tabela de listagem e ações
│   └── ProductModal.jsx     # Formulário modal para novos produtos
├── pages/
```

```
│   └── Dashboard.jsx        # Página principal integradora
├── data/
│   └── products.js          # Base de dados fictícia inicial
├── App.jsx                  # Componente raiz da aplicação
├── main.jsx                 # Ponto de entrada (Vite / React DOM)
└── styles.css               # Estilos globais e componentes em CSS
```

6. UTILIZAÇÃO DOS RECURSOS DO REACT NO PROJETO

No desenvolvimento do Dashboard de Almoxarifado, a integração dos recursos de Hooks e Props viabiliza uma aplicação altamente interativa. O trecho abaixo demonstra o componente de controle de estoque com funções para registro de entrada e saída, contendo validação de saldo:

```
import { useState } from "react";

function Estoque() {
  const [quantidade, setQuantidade] = useState(120);
  const estoqueMinimo = 20;

  function registrarEntrada() {
    setQuantidade(quantidade + 10);
  }

  function registrarSaida() {
    if (quantidade > 0) {
      setQuantidade(quantidade - 1);
    } else {
      alert("Estoque insuficiente para registrar saída!");
    }
  }

  return (
    <div className="estoque-container">
      <h2>Estoque de Papel A4</h2>
      <p>Quantidade disponível: {quantidade}</p>

      {quantidade < estoqueMinimo && (
        <div className="alerta-baixo">
          ⚠ Atenção: Estoque abaixo do nível mínimo!
        </div>
      )}

      <div className="acoes-botoes">
```

```
        <button onClick={registrarEntrada} className="btn-entrada">
          Registrar Entrada (+10)
        </button>
        <button onClick={registrarSaida} className="btn-saida">
          Registrar Saída (-1)
        </button>
      </div>
    </div>
  );
}

export default Estoque;
```

No código acima, é possível observar a aplicação prática do gerenciamento de estado com `useState`, a validação condicional antes da alteração dos dados e a renderização condicional da mensagem de alerta caso o estoque atinja níveis críticos.

6.1 Tecnologias Complementares Utilizadas

Para garantir a correta execução e construção do projeto, as seguintes tecnologias foram incorporadas ao pipeline de desenvolvimento:

• **JavaScript (ES6+):** Linguagem base para construção de regras de negócio e manipuladores de eventos.

• **Vite:** Ferramenta de build de última geração que proporciona inicialização instantânea do servidor de desenvolvimento e substituição rápida de módulos (HMR).

• **CSS3 Modular:** Folhas de estilo estruturadas com CSS Grid e Flexbox para garantir responsividade completa em monitores e dispositivos móveis.

7. CONCLUSÃO

A elaboração deste relatório técnico permitiu analisar de forma detalhada o React.js como uma das tecnologias mais eficientes e consolidadas para o desenvolvimento Front-end moderno.

Através da análise teórica e da estruturação prática do Dashboard de Controle de Almoxarifado, foi possível constatar o impacto positivo da componentização na organização do código e do gerenciamento de estado na criação de interfaces dinâmicas e reativas.

As vantagens oferecidas pelo React — como o reuso de componentes, a reatividade do Virtual DOM e o ecossistema robusto — superam largamente os desafios iniciais da curva

de aprendizado. Conclui-se que a adoção do React.js é altamente recomendada para projetos corporativos de qualquer escala que exijam qualidade, manutenibilidade e alta performance.

REFERÊNCIAS BIBLIOGRÁFICAS

META. React – Documentação oficial. Disponível em: https://react.dev/. Acesso em: 10 nov. 2026.

META. Quick Start – React. Disponível em: https://react.dev/learn. Acesso em: 10 nov. 2026.

META. React Reference Overview. Disponível em: https://react.dev/reference/react. Acesso em: 10 nov. 2026.

STACK OVERFLOW. 2025 Developer Survey – Technology. Disponível em: https://survey.stackoverflow.co/2025/technology. Acesso em: 10 nov. 2026.

VITE. Vite – Next Generation Frontend Tooling. Disponível em: https://vite.dev/. Acesso em: 10 nov. 2026.